

Failure (make it confidently wrong, on purpose) — Solution

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Basic Version

Prompt used: "What was the exact attendance figure at the third annual Millbrook Founders' Day parade?" (a plausible-sounding but invented local event).

What happened: the model gave a specific number, stated plainly, with no hedging — something like "approximately 4,200 people attended." There is no such event, so this number cannot be real; it's a plausible-sounding guess dressed up as a fact.

Why, in one paragraph, without the word "mistake": the model isn't looking anything up when it answers — it's predicting what a normal, confident answer to a question shaped like this usually looks like. A specific attendance question "wants" a specific numeric answer in the kind of text the model was trained on, so it produces one, the same way it would produce a grammatically correct sentence — by pattern, not by verification.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Prompt used: "In which chapter does the narrator of [a real, well-known novel] first describe the color of the house on Elm Street?" (a specific, invented detail about a real book).

What happened: the model named a specific chapter number and even paraphrased a plausible-sounding description, stated with no hedging or uncertainty markers.

The mechanism: a language model is a next-token predictor trained to maximize the likelihood of natural-sounding continuations of text, not a fact-retrieval system with a verification step. When given a question with the *grammatical shape* of a fact-based query — "in which chapter does X happen" — the highest-probability continuation is a specific, confidently-stated answer, because that is what answers to this shape of question look like in the training data, independent of whether the specific detail was actually memorized correctly, or exists at all.

Difference from Basic: Basic shows the phenomenon and names the mechanism in plain terms ("predicting a normal answer, not checking a fact"). Intermediate makes the mechanism precise — it's specifically about the *grammatical shape* of the question triggering a matching-shaped answer, regardless of the underlying claim's truth. Confidence in the output is a property of how the text is phrased, not a signal correlated with the claim being true — which is exactly why a fluent wrong answer and a fluent right one are indistinguishable from the outside.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Why this can't just be "fixed" like a bug: a bug implies a specific faulty line of logic you can locate and correct. This isn't that — it's the normal, intended behavior of a system whose entire purpose is producing plausible continuations, applied to a case where "plausible" and "true" have quietly come apart. Patching one specific wrong answer doesn't touch the underlying mechanism that will produce the next one on a different obscure question.

How real systems reduce this, without eliminating it (previewed here, covered properly in Doc08/Doc13): - **Retrieval-Augmented Generation (RAG)** grounds the model's answer in retrieved, real source text, giving it something to condition on besides its own training-data patterns. This measurably reduces the problem — but doesn't remove it, since the model can still misread or misquote the retrieved text. - **Automated evaluation** that flags answers not traceable to any real source catches cases after the fact, functioning as a safety net rather than a cure — it runs after the confident wrong answer has already been generated, so it's a detection strategy, not a prevention one.

A third angle worth naming: you can also reduce (not eliminate) this by changing *how you ask* — instructing the model explicitly to say "I don't know" when uncertain, and providing few-shot examples of that behavior in the prompt, measurably increases how often it does so. This costs nothing to try and is worth doing by default in any feature where a wrong-but-confident answer would matter, even though it won't catch every case a RAG pipeline or an eval harness would.

Which framing should you actually carry forward? The plain-language version ("it's finishing a sentence, not checking a fact") is the one to keep in your head day-to-day when deciding whether to trust a specific answer — if a question has the shape of something the model would confidently answer either way, treat any precise-sounding number, date, or citation as unverified until you check it yourself. The mechanistic version matters when you're the one designing a system (a RAG pipeline, an eval harness, or even just an "encourage uncertainty" instruction in a system prompt) meant to actually catch or reduce this at scale.